TDS Archive

Deep Learning With Apache Spark — Part 2

Second part on a full discussion on how to do Distributed Deep Learning with Apache Spark. I will focus entirely on the DL pipelines library and how to use it from scratch. One of the things you will be seeing are Transfer Learning on a simple Pipeline, how to use pre-trained models to work with “small” amount of data and being able to predict things and more.

Favio Vázquez · Follow

Published in TDS Archive · 10 min read · May 11, 2018

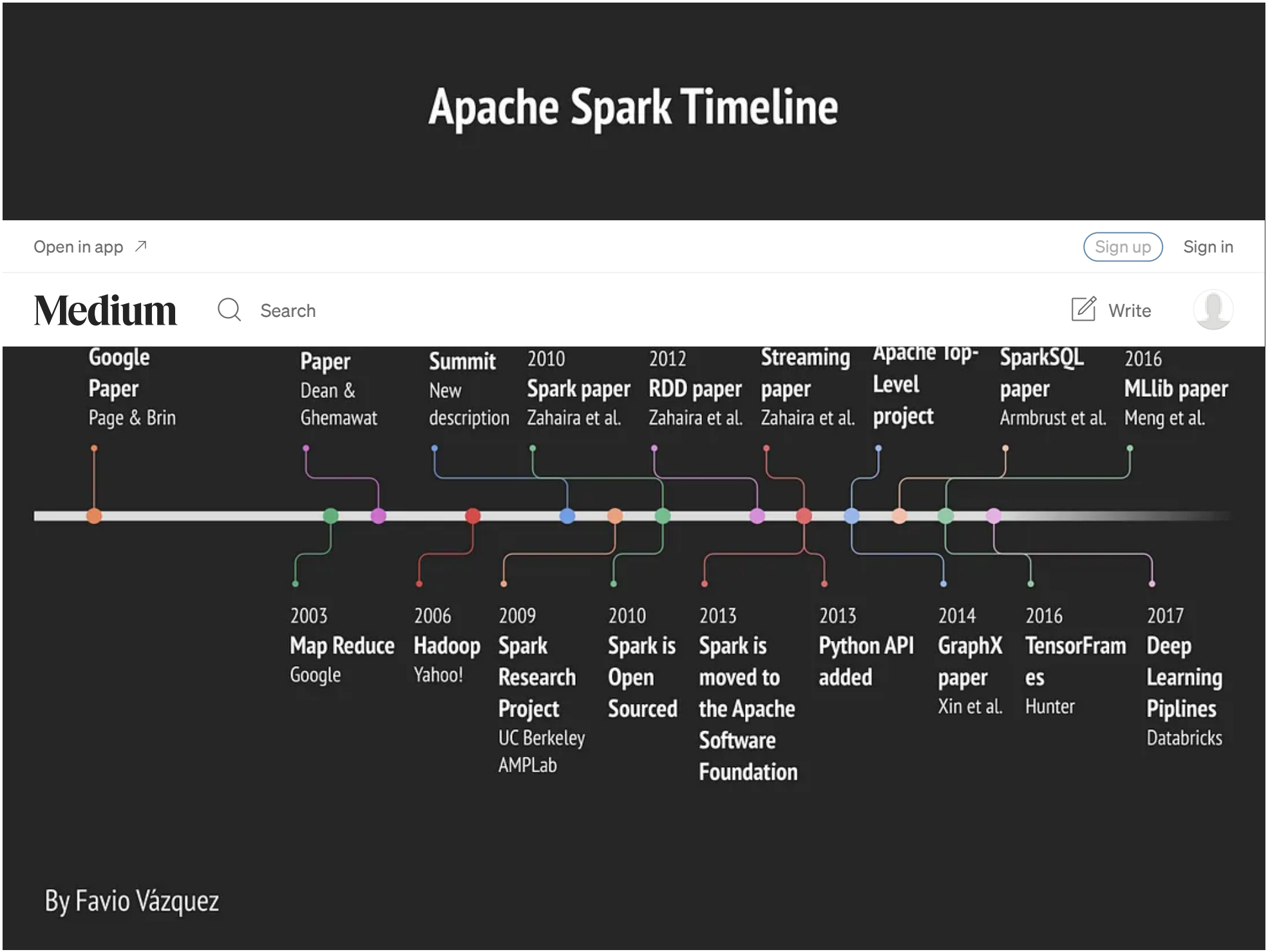
By my sister https://www.instagram.com/heizelvazquez/

Hi everyone and welcome back to learning :). In this article I'll continue the discussion on Deep Learning with Apache Spark. You can see the first part here.

In this part I will focus entirely on the DL pipelines library and how to use it from scratch.

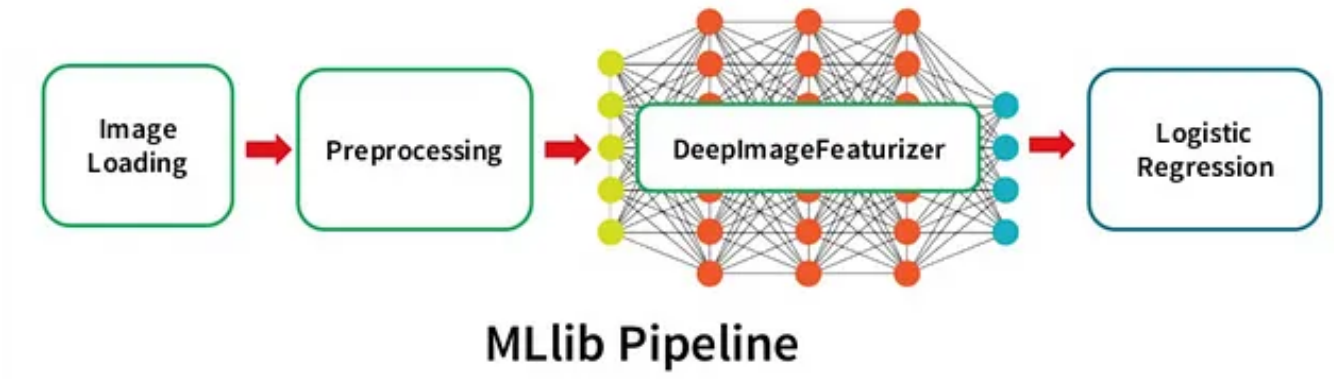
Apache Spark Timeline

The continuous improvements on Apache Spark lead us to this discussion on how to do Deep Learning with it. I created a detailed timeline of the development of Apache Spark until now to see how we got here.


Apache Spark Timeline
Open in app
Sign up
Sign in
Medium
Search
Write
Google Paper
Page & Brin
Paper
Dean & Ghemawat
Summit
New description
2010
Spark paper
Zahaira et al.
2012
RDD paper
Zahaira et al.
Streaming paper
Zahaira et al.
Apache Top-Level project
SparkSQL paper
Armbrust et al.
2016
MLlib paper
Meng et al.
2003
Map Reduce
Google
2006
Hadoop
Yahoo!
2009
Spark Research Project
UC Berkeley AMPLab
2010
Spark is Open Sourced
2013
Spark is moved to the Apache Software Foundation
2013
Python API added
2014
GraphX paper
Xin et al.
2016
TensorFrames
Hunter
2017
Deep Learning Piplines
Databricks
By Favio Vázquez

Soon I'll create an article with descriptions for this timeline but if you think there's something missing please let me know :)

Deep Learning Pipelines



Databricks

Deep Learning Pipelines is an open source library created by Databricks that provides high-level APIs for scalable deep learning in Python with Apache Spark.

databricks/spark-deep-learning

spark-deep-learning — Deep Learning Pipelines for Apache Spark

github.com

It is an awesome effort and it won't be long until is merged into the official API, so is worth taking a look of it.

Some of the advantages of this library compared to the ones that joins Spark with DL are:

- In the spirit of Spark and Spark MLlib, it provides easy-to-use APIs that enable deep learning in very few lines of code.
- It focuses on ease of use and integration, without sacrificing performace.
- It's build by the creators of Apache Spark (which are also the main contributors) so it's more likely for it to be merged as an official API than others.
- It is written in Python, so it will integrate with all of its famous libraries, and right now it uses the power of TensorFlow and Keras, the two main libraries of the moment to do DL.

Deep Learning Pipelines builds on Apache Spark's ML Pipelines for training, and with Spark DataFrames and SQL for deploying models. It includes high-level APIs for common aspects of deep learning so they can be done efficiently in a few lines of code:

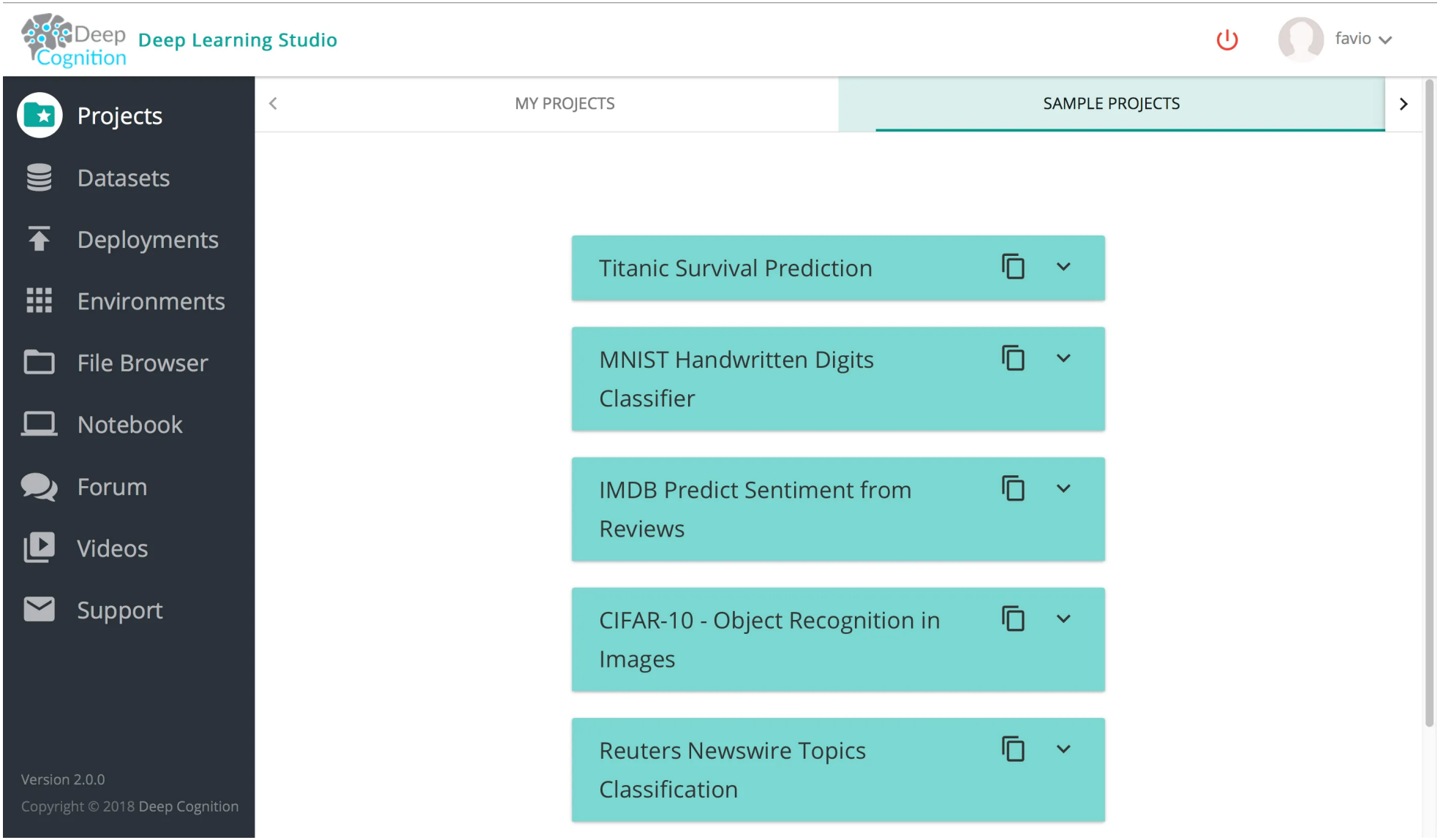
- Image loading
- Applying pre-trained models as transformers in a Spark ML pipeline
- Transfer learning
- Applying Deep Learning models at scale
- Distributed hyperparameter tuning **(next part)**
- Deploying models in DataFrames and SQL

I will describe each of these features in detail with examples. These examples comes from the official notebook by Databricks.

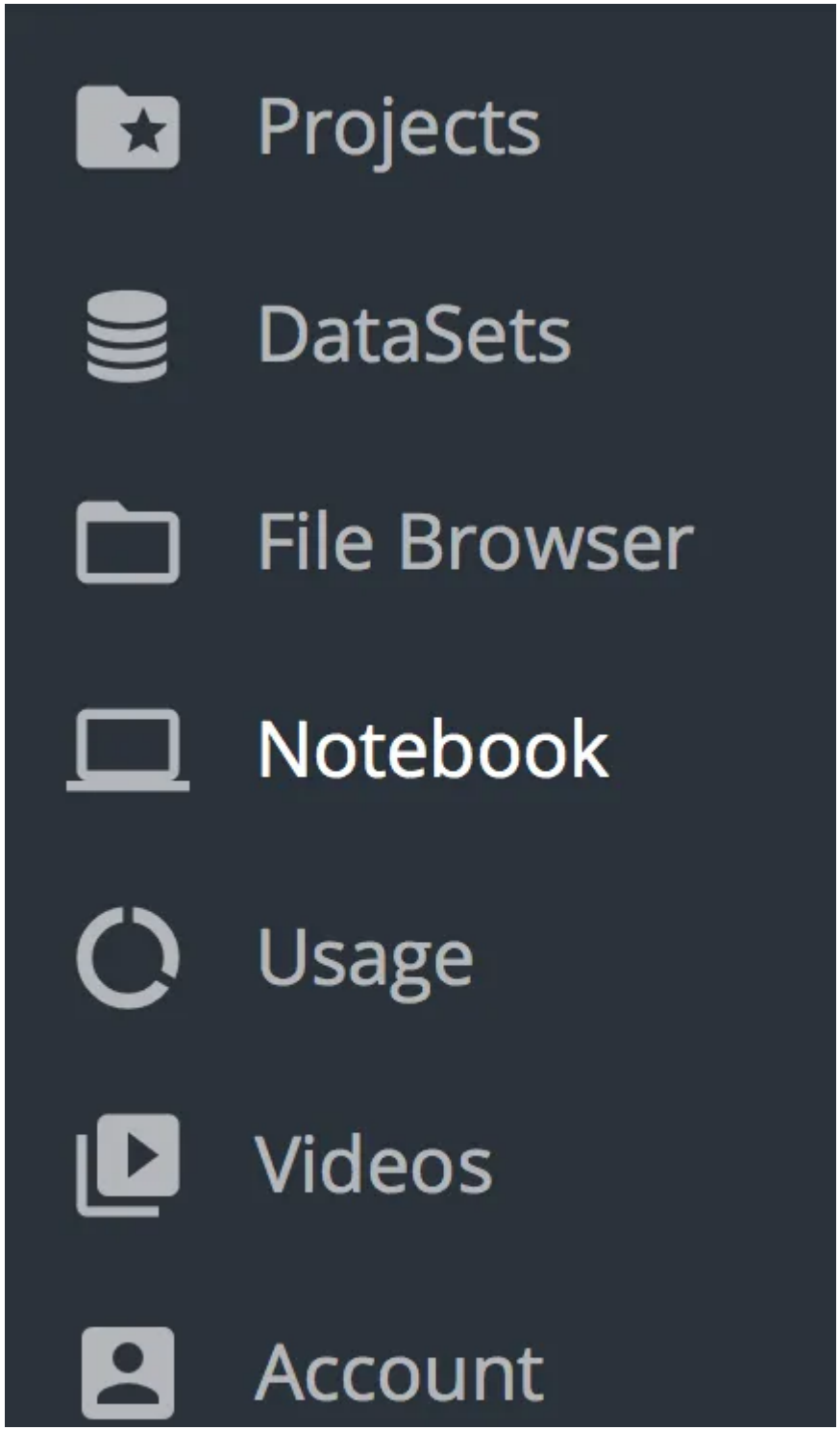
Apache Spark on Deep Cognition

To run and test the codes in this article you will need to create an account in Deep Cognition.

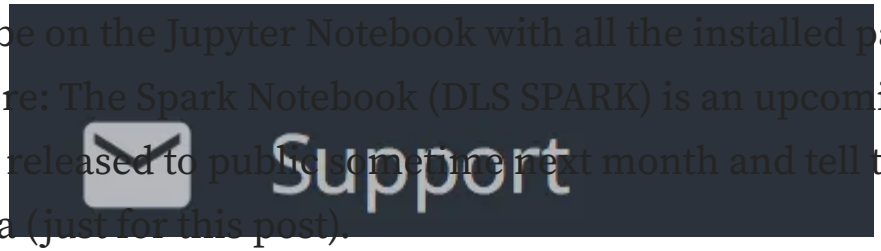
Is very easy and then you can access all of their features. When you log in this is what you should be seeing:

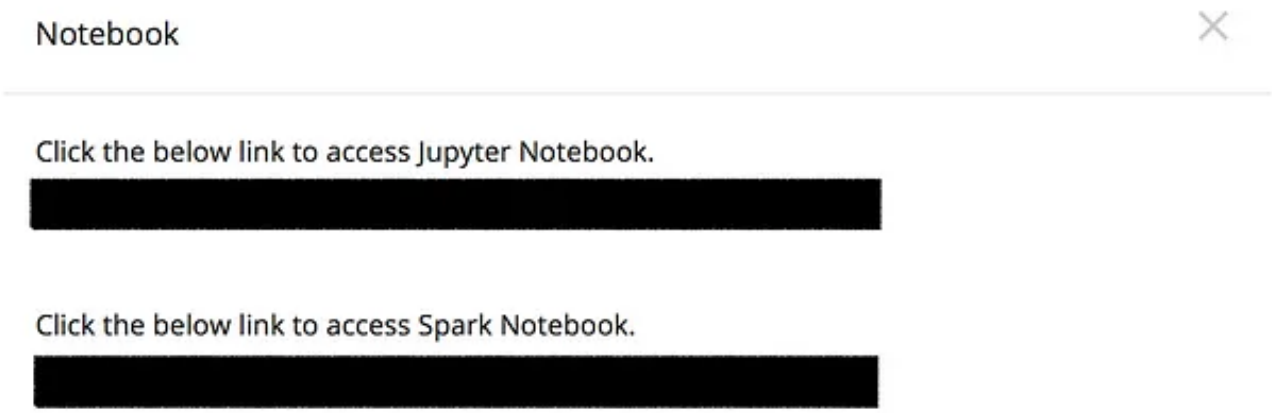


Now just click on the left part, the Notebook button:


Projects
DataSets
File Browser
Notebook
Usage
Videos
Account

And you will be on the Jupyter Notebook with all the installed packages :). Oh! A note here: The Spark Notebook (DLS SPARK) is an upcoming feature which will be released to public sometime next month and tell that it is still in private beta (just for this post).





You can download the full Notebook here to see all the code:

https://github.com/FavioVazquez/deep-learning-pyspark

Image Loading

The first step to applying deep learning on images is the ability to load the images. Deep Learning Pipelines includes utility functions that can load millions of images into a DataFrame and decode them automatically in a distributed fashion, allowing manipulation at scale. The new version of spark (2.3.0) has this ability too but we will be using the sparkdl library.

We will be using the archive of creative-commons licensed flower photos curated by TensorFlow to test this out. To get the set of flower photos, run these commands from the notebook (we will also create a sample folder):

```
!curl -O http://download.tensorflow.org/example_images/flower_photos.tgz
!tar xzf flower_photos.tgz
!mkdir flower_photos/sample
```

load_photos.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/33350294e31213ff761bf2ff51e25c4a

Let's copy some photos from the tulips and daisy folders to create a small sample of the photos.

```
!cp flower_photos/daisy/100080576_f52e8ee070_n.jpg flower_photos/sample/
!cp flower_photos/daisy/10140303196_b88d3d6cec.jpg flower_photos/sample/
!cp flower_photos/tulips/100930342_92e8746431_n.jpg flower_photos/sample/
```

create_sample.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/8ce726807f6074c05a779ee4e5e3a4d0

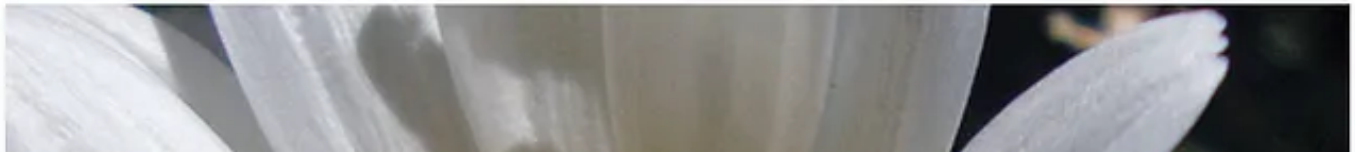
To take a look at these images on the notebook you can run this:

```python
import IPython.display as dp

# collect all .png files in ssample dir
fs = !ls flower_photos/sample/*.jpg

# create list of image objects
images = []
for ea in fs:
    images.append(dp.Image(filename=ea, format='png'))

# display all images
for ea in images:
    dp.display_png(ea)
```

see_images.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/efaa901f85b51c77d520595136a2cb52

You should be seeing this



```
from sparkdl import readImages

# Read images using Spark
image_df = readImages("flower_photos/sample/")
```

load_img_spark.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/85266329b7ef31411600f33c3b9eee1e

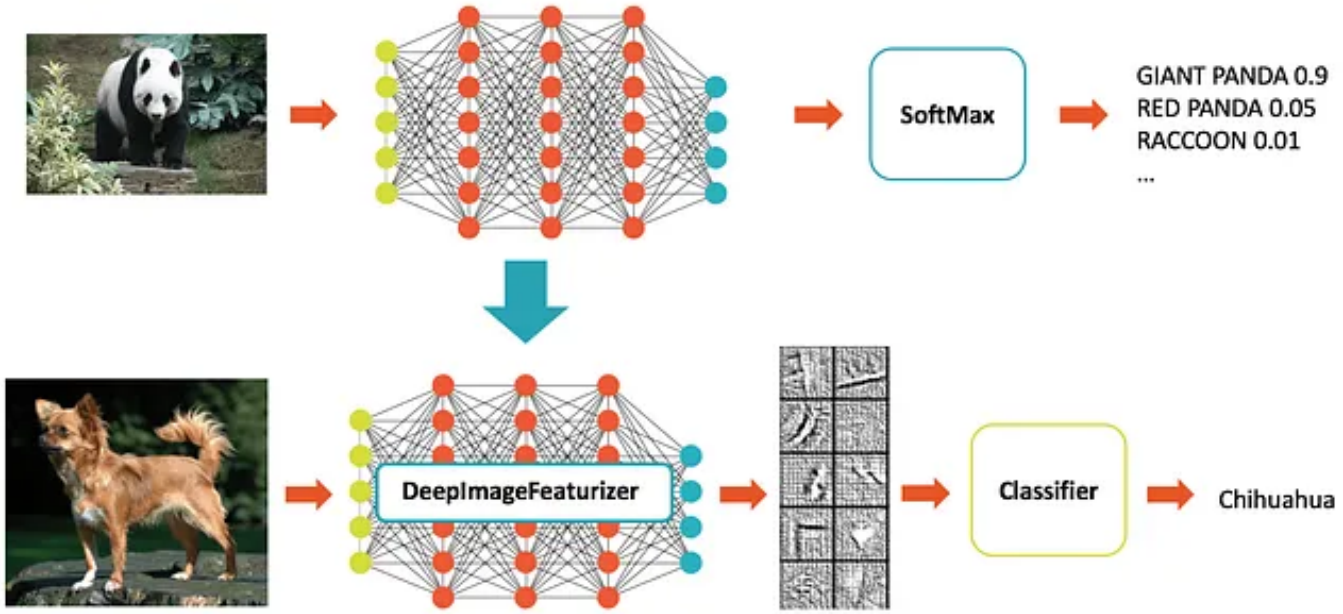
If we take a look at this dataframe we see that it spark created one column, called “image”.

```
image_df.show()

+--------------------+
|               image|
+--------------------+
|[file:/Users/favi...|
|[file:/Users/favi...|
```

```
|[file:/Users/favi...|
+--------------------+
```

The image column contains a string column contains an image struct with schema == ImageSchema.

Transfer learning



Databricks

Deep Learning Pipelines provides utilities to perform transfer learning on images, which is one of the fastest (code and run-time -wise) ways to start

using deep learning. Using Deep Learning Pipelines, it can be done in just several lines of code.

Deep Learning Pipelines enables fast transfer learning with the concept of a *Featurizer*. The following example combines the InceptionV3 model and logistic regression in Spark to adapt InceptionV3 to our specific domain. **The DeepImageFeaturizer automatically peels off the last layer of a pre-trained neural network and uses the output from all the previous layers as features for the logistic regression algorithm.** Since logistic regression is a simple and fast algorithm, this transfer learning training can converge quickly using far fewer images than are typically required to train a deep learning model from ground-up.

Firstly, we need to create training & test DataFrames for transfer learning.

```
from pyspark.ml.image import ImageSchema
from pyspark.sql.functions import lit
from sparkdl.image import imageIO

tulips_df = ImageSchema.readImages("flower_photos/tulips").withColumn("label", lit(1))
daisy_df = imageIO.readImagesWithCustomFn("flower_photos/daisy", decode_f=imageIO.PIL_decode).wit
tulips_train, tulips_test, _ = tulips_df.randomSplit([0.1, 0.05, 0.85])  # use larger training se
daisy_train, daisy_test, _ = daisy_df.randomSplit([0.1, 0.05, 0.85])     # use larger training se
train_df = tulips_train.unionAll(daisy_train)
test_df = tulips_test.unionAll(daisy_test)

```

```
# Under the hood, each of the partitions is fully loaded in memory, which may be expensive.
# This ensure that each of the paritions has a small size.
train_df = train_df.repartition(100)
test_df = test_df.repartition(100)
```

datasets_transfer.py hosted with ❤ by GitHub view raw

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline
from sparkdl import DeepImageFeaturizer

featurizer = DeepImageFeaturizer(inputCol="image", outputCol="features", modelName="InceptionV3")
lr = LogisticRegression(maxIter=10, regParam=0.05, elasticNetParam=0.3, labelCol="label")
p = Pipeline(stages=[featurizer, lr])

p_model = p.fit(train_df)
```

class_transfer.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/96e13301b6286eb7b52f34faedce4c24

Let's see how well the model does:

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

tested_df = p_model.transform(test_df)
evaluator = MulticlassClassificationEvaluator(metricName="accuracy")
print("Test set accuracy = " + str(evaluator.evaluate(tested_df.select("prediction", "label"))))
```

eval_transfer.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/27fa7de28011d41b192d723a185a9b87

```
Test set accuracy = 0.9753086419753086
```

Not so bad for an example and with no tunning at all!

We can take look at where we are making mistakes:

```
from pyspark.sql.types import DoubleType
from pyspark.sql.functions import expr
from pyspark.sql.functions import *
from pyspark.sql.types import *

def _p1(v):
    return float(v.array[1])y
take_one = udf(_p1, DoubleType())

df = tested_df.withColumn("p", take_one(tested_df.probability))
wrong_df = df.orderBy(expr("abs(p - label)"), ascending=False)
wrong_df.select("image.origin", "p", "label").show(10)
```

errors_transfer.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/dcd72fe4f0f4204736d46ba57112cb97

Applying Deep Learning models at scale

Deep Learning Pipelines supports running pre-trained models in a distributed manner with Spark, available in both batch and streaming data

processing.

It houses some of the most popular models, enabling users to start using deep learning without the costly step of training a model. The predictions of the model, of course, is done in parallel with all the benefits that come with Spark.

In addition to using the built-in models, users can plug in Keras models and TensorFlow Graphs in a Spark prediction pipeline. This turns any single-node models on single-node tools into one that can be applied in a distributed fashion, on a large amount of data.

The following code creates a Spark prediction pipeline using InceptionV3, a state-of-the-art convolutional neural network (CNN) model for image classification, and predicts what objects are in the images that we just loaded.

```
from sparkdl import DeepImagePredictor
# Read images using Spark
image_df = ImageSchema.readImages("flower_photos/sample/")

predictor = DeepImagePredictor(inputCol="image", outputCol="predicted_labels", modelName="Inceptic
predictions_df = predictor.transform(image_df)
```

models_scale_1.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/bbe4ab8787f4bd4a7f6bd8b8a8bc3521

Let's take a look to the predictions dataframe:

```
predictions_df.select("predicted_labels").show(truncate=False,n=3)
```

```
+----------------+
|predicted_labels|
|                |
+----------------+
|[[n03930313, picket_fence, 0.1424783], [n11939491, daisy,
0.10951301], [n03991062, pot, 0.04505], [n02206856, bee, 0.03734662],
[n02280649, cabbage_butterfly, 0.019011213], [n13133613, ear,
0.017185668], [n02219486, ant, 0.014198389], [n02281406,
sulphur_butterfly, 0.013113698], [n12620546, hip, 0.012272579],
[n03457902, greenhouse, 0.011370744]]            |

|[[n11939491, daisy, 0.9532104], [n02219486, ant, 6.175268E-4],
[n02206856, bee, 5.1203516E-4], [n02190166, fly, 4.0093894E-4],
[n02165456, ladybug, 3.70687E-4], [n02281406, sulphur_butterfly,
3.0587992E-4], [n02112018, Pomeranian, 2.9011074E-4], [n01795545,
black_grouse, 2.5667972E-4], [n02177972, weevil, 2.4875381E-4],
[n07745940, strawberry, 2.3729511E-4]]|

|[[n11939491, daisy, 0.89181453], [n02219486, ant, 0.0012404523],
[n02206856, bee, 8.13047E-4], [n02190166, fly, 6.03804E-4],
[n02165456, ladybug, 6.005444E-4], [n02281406, sulphur_butterfly,
5.32096E-4], [n04599235, wool, 4.6653638E-4], [n02112018, Pomeranian,
4.625338E-4], [n07930864, cup, 4.400617E-4], [n02177972, weevil,
4.2434104E-4]]                    |
+----------------+
only showing top 3 rows
```

Notice that the `predicted_labels` column shows "daisy" as a high probability class for all of sample flowers using this base model, **for some reason the tulip was closer to a picket fence than to a flower (maybe because of the background of the photo).**

However, as can be seen from the differences in the probability values, the neural network has the information to discern the two flower types. Hence our transfer learning example above was able to properly learn the differences between daisies and tulips starting from the base model.

Let's see how well our model discern the type of the flower:

```
df = p_model.transform(image_df)
# 100930342_92e8746431_n.jpg not a daisy
df.select("image.origin",(1-take_one(df.probability)).alias("p_daisy")).show(truncate=False)

+-----------------------------------------------+--------------------+
|origin                                         |p_daisy             |
+-----------------------------------------------+--------------------+
|.../100930342_92e8746431_n.jpg                 |0.016760347798379538|
|.../10140303196_b88d3d6cec.jpg                 |0.9704259547739851  |
|.../100080576_f52e8ee070_n.jpg                 |0.9705190124824862  |
+-----------------------------------------------+--------------------+
```

error_scale.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/271c069453b5917d85aeec0001d54624

For Keras users

For applying Keras models in a distributed manner using Spark, `KerasImageFileTransformer` works on TensorFlow-backed Keras models. It

- Internally creates a DataFrame containing a column of images by applying the user-specified image loading and processing function to the input DataFrame containing a column of image URIs
- Loads a Keras model from the given model file path
- Applies the model to the image DataFrame

To use the transformer, we first need to have a Keras model stored as a file. For this notebook we'll just save the Keras built-in InceptionV3 model instead of training one.

```
from keras.applications import InceptionV3

model = InceptionV3(weights="imagenet")
model.save('model-full.h5')  # saves to the local filesystem
```

save_keras.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/bc7d280cd98a7112cb96f13cded20259

Now we will create a Keras transformer but first we will preprocess the images to work with it

```
from keras.applications.inception_v3 import preprocess_input
from keras.preprocessing.image import img_to_array, load_img
import numpy as np
from pyspark.sql.types import StringType
from sparkdl import KerasImageFileTransformer

def loadAndPreprocessKerasInceptionV3(uri):
  # this is a typical way to load and prep images in keras
    image = img_to_array(load_img(uri, target_size=(299, 299)))  # image dimensions for Inceptior
    image = np.expand_dims(image, axis=0)
    return preprocess_input(image)

transformer = KerasImageFileTransformer(inputCol="uri", outputCol="predictions",
                                        modelFile='model-full.h5',  # local file path for model
                                        imageLoader=loadAndPreprocessKerasInceptionV3,
                                        outputMode="vector")
```

keras_transformer.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/b1a43d8611e1fd2db9a3c61742156e97

We will read now the images and load them into a Spark Dataframe and them use our transformer to apply the model into the images:

```
fs = !ls flower_photos/sample/*.jpg
uni_df = spark.createDataFrame(fs, StringType()).toDF("uni")
```

```
uri_df = sparkdl.createDataFrame(fns, "string").toDF("uri")
keras_pred_df = transformer.transform(uri_df)
```

keras_transf_pred.py hosted with ❤ by GitHub view raw

If we take a look of this dataframe with predictions we see a lot of informations, and that's just the probability of each class in the InceptionV3 model.

Working with general tensors

Deep Learning Pipelines also provides ways to apply models with tensor inputs (up to 2 dimensions), written in popular deep learning libraries:

- TensorFlow graphs
- Keras models

In this article we will focus only in the Keras models. The `KerasTransformer` applies a TensorFlow-backed Keras model to tensor inputs of up to 2 dimensions. It loads a Keras model from a given model file path and applies the model to a column of arrays (where an array corresponds to a Tensor), outputting a column of arrays.

```
from sparkdl import KerasTransformer
from keras.models import Sequential
from keras.layers import Dense
import numpy as np

```

```
# Generate random input data
num_features = 10
num_examples = 100
input_data = [{"features" : np.random.randn(num_features).astype(float).tolist()} for i in range(
schema = StructType([ StructField("features", ArrayType(FloatType()), True)])
input_df = spark.createDataFrame(input_data, schema)

# Create and save a single-hidden-layer Keras model for binary classification
# NOTE: In a typical workflow, we'd train the model before exporting it to disk,
# but we skip that step here for brevity
model = Sequential()
model.add(Dense(units=20, input_shape=[num_features], activation='relu'))
model.add(Dense(units=1, activation='sigmoid'))
model_path = "simple-binary-classification"
model.save(model_path)

# Create transformer and apply it to our input data
transformer = KerasTransformer(inputCol="features", outputCol="predictions", modelFile=model_path
final_df = transformer.transform(input_df)
```

keras_model.py hosted with ❤ by GitHub view raw

```
|  [0.5232896]|[-0.031451378, -1...|
| [0.45858437]|[0.9310042, -1.77...|
| [0.49794272]|[-0.37061003, -1....|
|  [0.2543479]|[0.41954428, 1.88...|
+-------------+--------------------+
only showing top 20 rows
```

Deploying Models in SQL

One way to productionize a model is to deploy it as a Spark SQL User Defined Function, which allows anyone who knows SQL to use it. Deep Learning Pipelines provides mechanisms to take a deep learning model and *register* a Spark SQL User Defined Function (UDF). In particular, Deep Learning Pipelines 0.2.0 adds support for creating SQL UDFs from Keras models that work on image data.

The resulting UDF takes a column (formatted as a image struct "SpImage") and produces the output of the given Keras model; e.g. for Inception V3, it produces a real valued score vector over the ImageNet object categories.

```
from keras.applications import InceptionV3
from sparkdl.udf.keras_image_model import registerKerasImageUDF

registerKerasImageUDF("inceptionV3_udf", InceptionV3(weights="imagenet"))
```

dl_sql_1.py hosted with ♥ by GitHub view raw

https://gist.github.com/FavioVazquez/3a36edf25a289f4ee31cff1bf3857467

In Keras workflows dealing with images, it's common to have preprocessing steps before the model is applied to the image. If our workflow requires preprocessing, we can optionally provide a preprocessing function to UDF registration. The preprocessor should take in a filepath and return an image array; below is a simple example.

```
from keras.applications import InceptionV3
from sparkdl.udf.keras_image_model import registerKerasImageUDF

def keras_load_img(fpath):
    from keras.preprocessing.image import load_img, img_to_array
    import numpy as np
    img = load_img(fpath, target_size=(299, 299))
    return img_to_array(img).astype(np.uint8)

registerKerasImageUDF("inceptionV3_udf_with_preprocessing", InceptionV3(weights="imagenet"), kera
```

dl_sql_2.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/a02094a5848ab1f7e42ce52820a09fbe

Once a UDF has been registered, it can be used in a SQL query:

```
from pyspark.ml.image import ImageSchema

image_df = ImageSchema.readImages("flower_photos/sample/")
image_df.registerTempTable("sample_images")

df = spark.sql("SELECT inceptionV3_udf_with_preprocessing(image) as predictions from sample_images
```

dl_sql_3.py hosted with ❤ by GitHub view raw

https://gist.github.com/FavioVazquez/af566a98d19952eb0b61938c4752f7dc

This is very powerful. Once a data scientist builds the desired model, Deep Learning Pipelines makes it simple to expose it as a function in SQL, so anyone in their organization can use it — data engineers, data scientists, business analysts, anybody.

```
sparkdl.registerKerasUDF("awesome_dl_model",
"/mymodels/businessmodel.h5")
```

Next, any user in the organization can apply prediction in SQL:

```
SELECT image, awesome_dl_model(image) label FROM images
WHERE contains(label, “Product”)
```

In the next part I'll discuss Distributed Hyperparameter Tuning with Spark, and will try new models and examples :).

If you want to contact me make sure to follow me on twitter:

Favio Vázquez (@FavioVaz) | Twitter

The latest Tweets from Favio Vázquez (@FavioVaz). Data Scientist. Physicist and computational engineer. I have a...

twitter.com

and LinkedIn:

Machine Learning | Deep Learning | Pyspark | Apache Spark

Towards Data Science

Published in TDS Archive

815K Followers · Last published Feb 4, 2025

Follow

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.

Written by Favio Vázquez

9.2K Followers · 17 Following

Follow

Data scientist, physicist and computer engineer. Love sharing ideas, thoughts and contributing to Open Source in Machine Learning and Deep Learning ;).

Responses (7)

Write a response

What are your thoughts?

Munisha Bansal
Jan 9, 2020

I am trying to use a keras model (a deep NN for regression) on Spark cluster. the solution here given is using sparkdl and Kerastransformer. I installed sparkdl but there are many import issues and not able to solve them. Has anyone else faced this ?

Reply

Samik R
Nov 30, 2018

Can the DataBricks package be used with Scala as well? I kow that the BigDL package can be.

Reply

Phil Scruggs
Nov 27, 2018

After creating an account I am unable to get the SparkContext to load on Deep Cognition… Curious if anyone else has actually tried this lately (11/27/2018)

Reply

See all responses

More from Favio Vázquez and TDS Archive

In Ciencia y Datos by Favio Vázquez

Cómo usar PySpark en tu computadora

He descubierto que es un poco difícil comenzar con Apache Spark (esto se...

Apr 30, 2018 151 5

In TDS Archive by Shaw Talebi

5 AI Projects You Can Build This Weekend (with Python)

From beginner-friendly to advanced

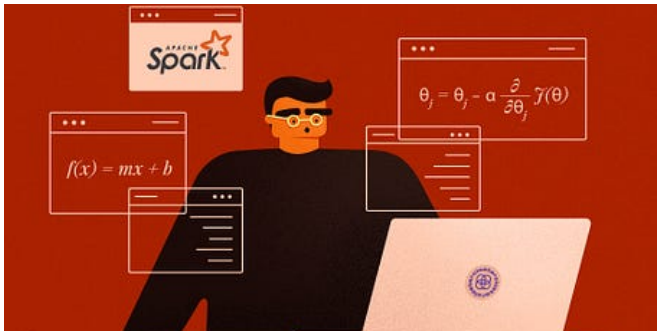
Oct 9, 2024 4.6K 75

In TDS Archive by Mauro Di Pietro

GenAI with Python: Build Agents from Scratch (Complete Tutorial)

with Ollama, LangChain, LangGraph (No GPU, No APIKEY)

Sep 29, 2024 2.7K 47



In TDS Archive by Favio Vázquez

Your First Apache Spark ML Model

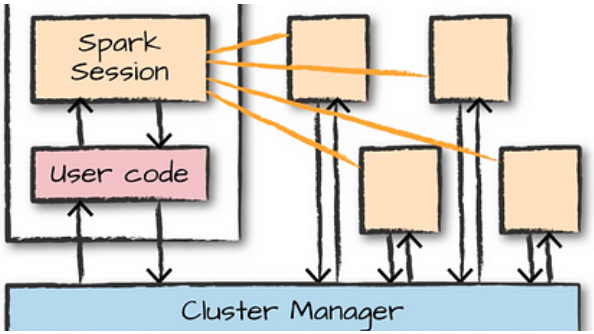
How to build a basic machine learning model with Apache Spark and Python.

Jun 17, 2020 452 5

See all from Favio Vázquez

See all from TDS Archive

Recommended from Medium

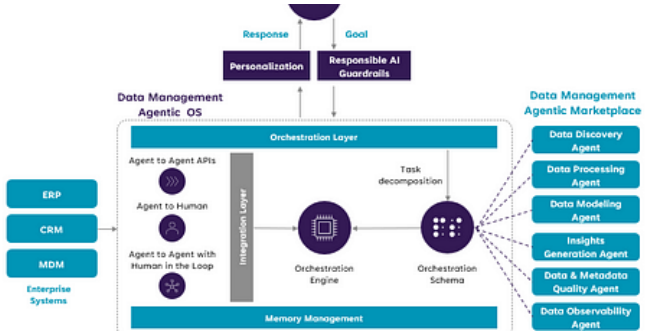


In Towards Dev by Prem Vishnoi(cloudvala)

Apache Spark Architecture :A Deep Dive into Big Data Processing

Agenda

Feb 6 88 1

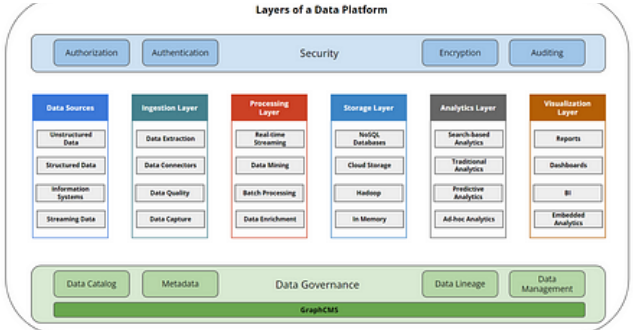


In AI Advances by Debmalya Biswas

Agentic AI for Data Engineering

Reimagining Enterprise Data Management leveraging AI Agents

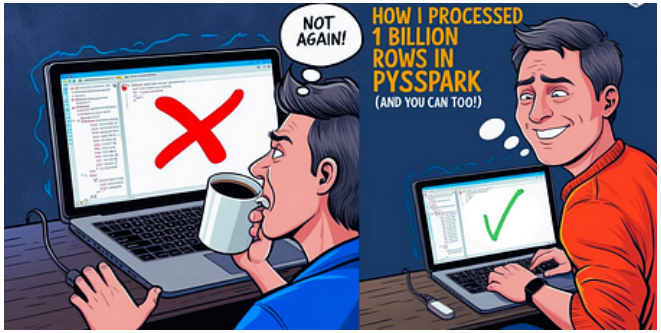
Mar 24 598 15



Joanna

All you need to know to be a data product manager (platform...

I've been working as a data PM for a few years. In this article, I'd like to share more...



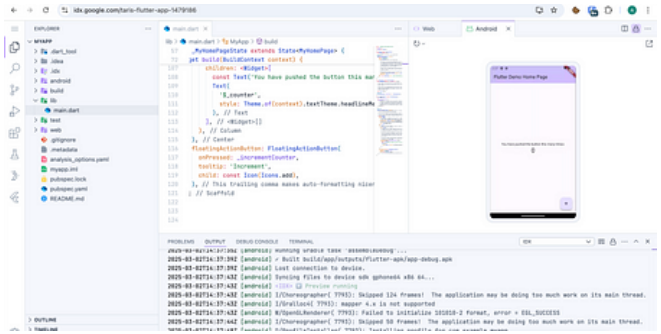
Varsha C Bendre

How I processed ONE billion rows in PySpark without crashing (and Yo...

Ever tried running a PySpark job on 1 billion rows, only to watch it crash and burn?

Mar 6 1

Feb 19 50



In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

Mar 12 3.2K 190

Ani

Apache Spark Learning Path for a Budding Data Engineer

Introduction

Oct 15, 2024 13

See more recommendations